Interview Preparation Guide for Rohit Jangir Target: Interview with Anupa Kamath (Director of Engineering, Fortanix)

---------- 1. System Design & Architecture ---------- Q: How would you design a secure, scalable microservices-based system? A: In my current role at Nagravision, I designed and developed several microservices using Java and Spring Boot. Each service had a clearly defined responsibility and communicated asynchronously via RabbitMQ to reduce coupling and improve reliability. For scalability, we containerized the services using Docker and orchestrated them with Kubernetes, enabling auto-scaling and blue-green deployments. From a security standpoint, we used JWT-based authentication for APIs and AWS IAM roles for resource access. We also applied rate-limiting and SSL termination through NGINX. For observability, we integrated logs into ELK and added Prometheus metrics to track latency and throughput. This approach improved deployment speed by ~30% and system performance by 20%.

---------- 2. Security & Data Protection ---------- Q: How do you ensure data security in a cloud environment? A: I start with a “defense-in-depth” mindset. In AWS, I use IAM policies with least privilege, encrypt all data at rest using KMS, and enforce TLS for data in transit. At the application layer, I use JWT authentication and role-based access control. For secrets management, I rely on AWS Secrets Manager or Kubernetes secrets instead of environment variables. In one of our projects, we implemented a secure file encryption workflow using AES for data stored in S3 — this was inspired by my academic project “Securing Data in Cloud using QAES.” I also believe in strong observability — continuous monitoring of logs and access patterns helps detect anomalies early.

---------- 3. Cloud, Deployment & Efficiency ---------- Q: How do you manage deployments and optimize cost in cloud? A: I’ve worked extensively with containerized deployments using Docker and Kubernetes. We created a CI/CD pipeline through Jenkins, automating build, test, and deployment phases. One impactful project was implementing a multi-tenant architecture — it allowed multiple customers to share the same AWS infrastructure while maintaining isolation at the data and configuration levels. This design cut cloud costs significantly without compromising performance. I also monitor instance utilization metrics and right-size compute resources periodically to control spend.

---------- 4. Observability & Reliability ---------- Q: How do you handle debugging and observability in distributed systems? A: We use structured logging and correlation IDs across microservices so we can trace a request end-to-end. Metrics like latency, error rate, and queue depth are tracked in Prometheus and visualized in Grafana. During incidents, I rely on log aggregation (ELK stack) and distributed tracing (OpenTelemetry/Jaeger) to identify bottlenecks. This observability setup helped us detect performance regressions early and reduce MTTR by around 25%.

---------- 5. Testing & Quality Mindset ---------- Q: What’s your approach to testing microservices? A: I follow a pyramid strategy — unit tests using JUnit/Mockito, integration tests with Spring Boot Test, and end-to-end validations via Cucumber. For messaging, I simulate RabbitMQ queues in integration tests to ensure event delivery consistency. Our CI/CD pipeline executes these tests before deployment, and we use Canary releases for safe rollouts. This test-first approach has improved our release stability and reduced customer-reported defects.

---------- 6. Leadership & Ownership ---------- Q: Tell me about a time you took

ownership and solved a critical issue. A: During a production deployment, we faced high latency in one of our data migration microservices. I took ownership, profiled the service, and discovered inefficient queries in MongoDB. I optimized the indexes and refactored the data fetch logic, improving migration speed from ~5,000 to 15,000 accounts/hour. This not only stabilized the deployment but also earned me a Bravo Award for problem-solving and ownership.

---------- 7. Scalability & Performance ---------- Q: How did you achieve the 20% performance improvement you mentioned? A: The improvement came from optimizing the data flow between microservices. We previously had synchronous REST calls that caused blocking under load. I redesigned these interactions using RabbitMQ, turning them into asynchronous events. This reduced request latency and prevented cascading failures under heavy traffic. Additionally, we optimized database queries and added caching via Redis for frequently accessed data.

---------- 8. Behavioral & Teamwork ---------- Q: How do you handle disagreements in design discussions? A: I believe in data-driven decisions. I encourage open discussions, then back ideas with metrics, benchmarks, or proof-of-concepts. For example, when deciding between synchronous REST and messaging-based communication, I built a small performance benchmark showing how RabbitMQ could improve throughput. That helped the team align around facts, not opinions. It's important to balance technical correctness with team consensus and delivery timelines.

---------- 9. Continuous Improvement ---------- Q: What does operational excellence mean to you? A: Operational excellence means building systems that are reliable, observable, and continuously improving. It's about having clear SLIs and SLOs, automating repetitive tasks, and learning from incidents through postmortems. I've contributed to this by setting up deployment automation, adding metrics dashboards, and documenting runbooks that helped our support team resolve issues faster.

---------- 10. Learning & Growth ---------- Q: How do you keep yourself up to date? A: I follow engineering blogs from AWS, InfoQ, and Medium. Recently, I completed the AWS ML Foundation and Cybersecurity Foundation courses to strengthen my understanding of cloud and security principles. I also experiment with new languages like Rust for performance-critical use cases. Continuous learning helps me bring fresh perspectives and modern practices into my daily work.

---------- Summary Tips ---------- ■ Always connect your answers to real examples from your resume. ■ Use metrics ("improved by 20%", "reduced cost by X%"). ■ Structure each answer as: Context → Action → Result → Learning.